

6502life ROM Cheat Sheet

Geometry tables at \$E000-\$EE3F

Neighborhood Cell Indices

49 cells in spiral order. Cell n at address $n \times 1024$.

y	-3	-2	-1	x 0	+1	+2	+3
+3	48	44	36	25	29	37	45
+2	43	24	20	9	13	21	38
+1	35	19	8	1	5	14	30
0	28	12	4	0	2	10	26
-1	34	18	7	3	6	15	31
-2	42	23	17	11	16	22	39
-3	47	41	33	27	32	40	46

0=self, 1=N(0,+1), 2=E(+1,0), 3=S(0,-1), 4=W(-1,0)
5=NE, 6=SE, 7=SW, 8=NW, 9-12=N²E²S²W²

Cell Base Address

base = cell_index * \$0400

High byte = cell_index * 4 (low byte always \$00)

Cell 0 (self)	\$0000	Cell 5 (NE)	\$1400
Cell 1 (N)	\$0400	Cell 6 (SE)	\$1800
Cell 2 (E)	\$0800	Cell 7 (SW)	\$1C00
Cell 3 (S)	\$0C00	Cell 8 (NW)	\$2000
Cell 4 (W)	\$1000	Cell 9 (N ²)	\$2400

At runtime: ASL; ASL to get high byte from cell index.
Store in ZP pointer, use LDA (\$ptr),Y for indirect access.

ROM Table Map

Each row = 64 bytes. row_addr = \$E000 + row * 64.
Index by cell index (0-48). Out-of-range returns \$FF.

Address	Row	Operation
\$E000+64 <i>j</i>	0-48	Translate by cell <i>j</i>
\$EC40	49	Rotate 90° CW
\$EC80	50	Rotate 180°
\$ECC0	51	Rotate 270° CW
\$ED00	52	Reflect X (flip <i>y</i>)
\$ED40	53	Reflect Y (flip <i>x</i>)
\$ED80	54	<i>x</i> -coord + 3
\$EDC0	55	<i>y</i> -coord + 3
\$EE00	56	(<i>x</i> , <i>y</i>) → cell index

Translation (Vector Addition)

translate[j][i] = cell at $\vec{v}_i + \vec{v}_j$

Example: cell 1's east neighbor = cell at (0,+1) + (+1,0):

LDX #2; LDA \$E040,X → A=5 (NE)

Use: find neighbor-of-neighbor, compose directions,
check reachability (result ≠ \$FF means in range).

Rotation & Reflection

LDA \$EC40,X	Rot 90 CW	N→E→S→W
LDA \$EC80,X	Rot 180	N→S, E→W
LDA \$ECC0,X	Rot 270 CW	N→W→S→E
LDA \$ED00,X	Reflect X	N↔S
LDA \$ED40,X	Reflect Y	E↔W

Iterate cardinals:

LDX #1 / @loop: ... LDA \$EC40,X / TAX / CPX #1
/ BNE @loop

Coordinate Tables

LDX #cell; LDA \$ED80,X → $x + 3$ (range 0-6)
LDX #cell; LDA \$EDC0,X → $y + 3$ (range 0-6)

Origin (cell 0) = coordinate (3,3).

Reverse: \$EE00+($x + 3$)+($y + 3$)*8 → cell index.

LDA \$EE14 → 6 (offset (+1,-1): $x=4$, $y=2$, addr
\$EE00+4+16)

Oriented Registers (\$F0-\$F8)

Top 6 bits = cell index, auto-rotated by memory map-
per.

Bottom 2 bits = user data (preserved across rotations).

value = (cell_index << 2) | flags

Write a direction to \$F0; after interrupt + re-orientation,
\$F0 still points to the *same relative neighbor*.
No compass needed for rotation-invariant programs.

Note: \$F9 (PCHI) is also auto-rotated but is
reserved for the saved program counter high byte.

Compass (\$FA)

Requires hasCompass=true. \$FA = orientation << 2.

\$00 = 0° \$04 = 90° \$08 = 180° \$0C = 270°

To access absolute direction d (S=1,E=2,N=3,W=4):
local = ((d-1-orient+4) % 4) + 1

Lookup table (16 bytes, indexed by \$FA+($d - 1$)):

orient 0: 1 2 3 4
orient 1: 4 1 2 3
orient 2: 3 4 1 2
orient 3: 2 3 4 1

LDX \$FA; LDA tbl+2,X → local index for abs. North

Zero Page Layout (cell 0)

\$00-\$9F	Code / scratch
\$A0	Key input buffer
\$A1	Cell unique ID
\$A5	Type tag
\$A6	State length
\$A8-\$B7	State string (16 chars)
\$F0-\$F8	Oriented registers
\$F9	Saved PC high (oriented)
\$FA	Compass / saved PC low
\$FB	Saved P register
\$FC-\$FF	RNG (refreshed each interrupt)

Runtime Cell Access Pattern

When the target cell index is computed at runtime (e.g. from
compass resolution), use indirect addressing:

```
ASL                ; cell_index * 2
ASL                ; * 4 = addr high byte
STA $02            ; store high byte
LDA #$00
STA $01            ; low byte = 0
LDY #$A5           ; TYPE_TAG_OFFSET
LDA ($01),Y        ; read type tag
```

BRK Operations

BRK \$00	Reset	PC → 0, yield
BRK \$01--30	Swap	cell 0 ↔ cell <i>b</i>
BRK \$31--60	Copy	cell 0 → cell (<i>b</i> -48)

Swap is $O(1)$ (~49 cyc). Copy is $O(M)$ (~6000 cyc).
Fails silently if remaining cycles < cost.